Simplification & Refactoring Ideas

Supply Chain Command Center — June 16, 2026

Executive Summary

The codebase is well-structured at the domain level: clear router folders, centralized model_registry, make_pool in API tests, and shared_constants.yaml inheritance. Main debt is oversized monolith files, parallel legacy + new implementations (especially tuning), and mechanical rule drift in scripts.

What Is Already Good

- Domain router layout under api/routers/{inventory, forecasting, operations, platform, intelligence, core}/
- Reference split patterns: forecasting/tuning/ (13 sub-routers), customer_analytics/ (6 sub-routers)
- Test infrastructure: make_pool in tests/api/conftest.py
- ML centralization: tree estimators only in common/ml/model_registry.py
- Config inheritance via shared_constants.yaml _includes
- Frontend queries use fetchJson; zero : any in queries/
- Planning date centralized; no backward-compat config shims

P0 - High Impact, Low Risk

Opportunity	Evidence	Action
Split inv_planning_insights.py	1,807 LoC, 11 endpoints - largest router	Create api/routers/inventory/insights/ subpackage
Migrate query modules to fetchJson	8 modules still use raw fetch()	Route through queries/core.ts fetchJson
Extend raw-fetch guard	no-raw-fetch.test.ts watches only 4 files	Add 4 model-tuning tab files to WATCHED list
PROJECT_ROOT migration in scripts	~60 scripts use Path(__file__).parents[N]	Use from common.core.paths import PROJECT_ROOT
Barrel-mount inventory routers	18 separate include_router calls in main.py	Aggregate like customer_analytics/__init__.py
Barrel-mount domain routers	~80 manual imports in main.py (377 LoC)	Group by domain __init__.py - no URL changes

P1 - Medium Effort, Good Payoff

Opportunity	Evidence	Action
Retire triple tuning stack	lgbm_tuning, model_tuning, forecasting/tuning/ in parallel	Deprecate legacy routers; UI uses unified-model-tuning.ts
Split routers over 800 LoC	9 routers exceed limit (see reference table)	Split by feature area; share experiment CRUD base
Frontend tab size violations	8+ tabs/panels over 600 LoC	Extract orchestration into existing subpanels
Deduplicate comparison UI	6 comparison panels with shared layout	Single parameterized ExperimentComparisonPanel
inv_planning prefix cleanup	Full paths embedded in decorators	Use APIRouter(prefix=...) per CLAUDE.md
Inline hex chart colors	~170 hex literals across 40+ tab files	Migrate to useChartColors() / useThemeContext()
Chunked fact-table reads	Bare pd.read_sql in several scripts	Use read_sql_chunked() for fact tables
Consolidate inv-planning queries	14 separate frontend query files	Merge into inv-planning/ subfolder with barrel export

P2 - Larger Refactors

Opportunity	Evidence	Action
Split backtest_framework.py	1,776 LoC	Extract sampling, embargo, persistence
Split job_state + job_registry	3,162 LoC combined	Clarify read vs write surfaces
Consolidate run_backtest_*.py	10+ thin wrappers	Single --algorithm=id CLI
Decompose domains.py	813 LoC catch-all	Migrate stable routes; shrink catch-all
Split api/core.py	653 LoC	Move domain query builders out
Split largest scripts	run_backtest, generate_production_forecasts, load	Stage-based modules
SHAP panel duplication	ShapPanel vs DfuShapPanel	Shared chart + thin wrappers

Anti-Patterns Found

Rule	Scope	Notes
Routers >800 LoC	9 files	inv_planning_insights (1,807) is worst offender
Tabs/panels >600 LoC	8+ files	EnhancedComparisonPanel (991) is worst offender
date.today() in production	1 file	dashboard.py:52 - document as exception
Path(__file__).parents	~60 scripts	Use common.core.paths.PROJECT_ROOT
except Exception volume	~70 files	Heaviest in experiment routers
_row_to_dict duplicates	4 routers	Use common/core/sql_helpers.py
Raw fetch in tabs	4 files	model-tuning/ panels bypass fetchJson

Over- vs Under-Engineering

Over-Engineered	Under-Engineered
3 parallel tuning API stacks	scripts_base.py underused (~7/93 scripts)
6 comparison panel components	inv_planning_insights monolith (11 endpoints)
job_state + job_registry (3,162 LoC)	60 scripts reinvent ROOT paths
14 inv-planning query modules	no-raw-fetch guard covers 4 files only

Largest Files

Layer	File	Lines
Router	inv_planning_insights.py	1,807
Router	champion_experiments.py	1,272
Router	backtest_management.py	1,061
common/	backtest_framework.py	1,776
common/	job_state.py	1,734
Script	run_backtest.py	1,573
Script	generate_production_forecasts.py	1,541
Frontend	EnhancedComparisonPanel.tsx	991

Top 10 Quick Wins

1. Replace raw fetch() with fetchJson

8 query modules - ~1 hour, no URL changes

2. Extend no-raw-fetch.test.ts

Cover 4 model-tuning tab files

3. Split inv_planning_insights.py

Highest LoC violation - split at endpoint boundaries

4. Barrel inventory routers

Remove 18 mount lines from main.py

5. PROJECT_ROOT in top scripts

run_backtest, load, generate_production_forecasts

6. Template inv_planning prefix

Prove pattern on one router, replicate to 17

7. Deprecate legacy tuning routers

New UI only through /model-tuning

8. Extract action-feed endpoint

Largest block inside insights router

9. Migrate FeatureLabPanel colors

Highest hex literal density (13)

10. Parameterize tuning API tests

Merge shared cases into test_unified_model_tuning

Recommended Starting Point

Highest ROI: split inv_planning_insights.py and migrate query modules to fetchJson. Both are mechanical, do not change URLs, and address the most violated CLAUDE.md rules. Second wave: retire the triple tuning stack and barrel-mount routers in main.py.